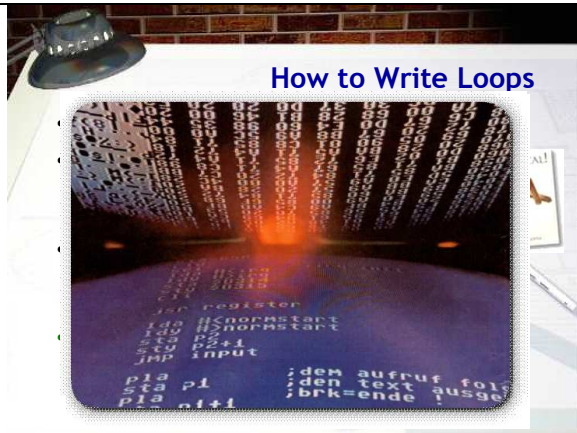




Slide 1

How to Write Loops

Duration: 00:01:30

Advance mode: Auto

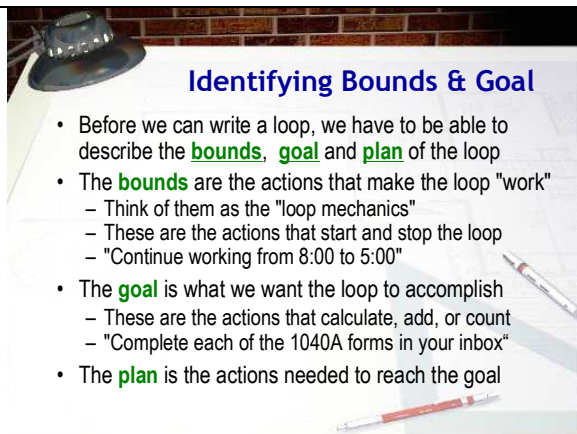
Notes:

Writing perfect code the first time is something of a "Holy Grail" among programmers. By that I mean that most programmers long to do it, but the vast majority consider its attainment to be the stuff of legend.

Writing loops is one area where programming errors often crop up. Several years ago, however, I happened upon a technique developed by Doug Cooper, the Berkeley professor of "Oh! Pascal" fame, for building loops. This technique really does increase your chances of building correct loops the first time, and it's worth the effort it takes to learn it.

Your textbook also has a similar strategy in section 6.1, titled "How to Write Loops" that you should read through. Having a strategy gives you some place to start and using a step-by-step approach, whether you use the techniques in your textbook or the Coopers method, will help you to avoid errors.

To start out with this section, open the file LoopBuilding.java in the ic14 project. Once it's open, run the program and snap a picture for me showing the console and the code open in Eclipse. Then, start a new section in your "in-class" lab document and paste the picture in for Exercise 1.



Slide 2

Identifying Bounds & Goal

Duration: 00:01:13

Advance mode: Auto

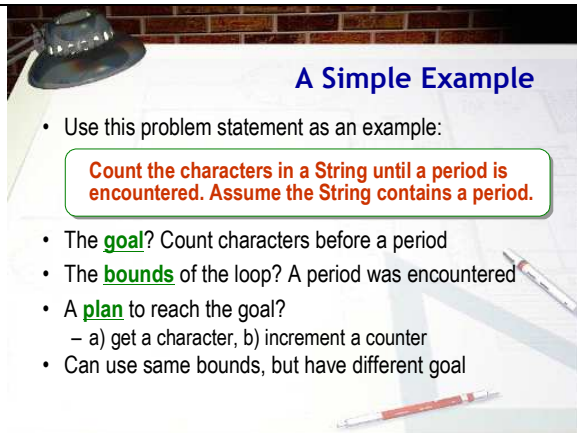
Notes:

The first step in building a successful loop is to be able to describe (and separate) the loop's bound from the loop's goal, and then come up with a plan for reaching your goal. So, what do we mean when we talk about the bounds and goal of a loop.

The bounds is the portion of the loop that makes it work "mechanically". Specifically, these are the actions or conditions that start and stop the loop. In Java, the bounds are the boolean condition that you'll put in parentheses after your while or for keyword. If you had a new job, for instance, the bounds would be something like your working hours—the time you start and stop.

The goal of the loop is the work that you want it to accomplish. This might be anything from summing a series of input values to drawing a series of dots on a graphics display. In your new job, the goal is what actual work your boss expects you to complete while you're working that 8 to 5 job.

Finally, there's the plan. The plan is the detailed actions you'll follow to both reach the bounds and, if possible, meet your goal.



A Simple Example

- Use this problem statement as an example:

Count the characters in a String until a period is encountered. Assume the String contains a period.

- The **goal**? Count characters before a period
- The **bounds** of the loop? A period was encountered
- A **plan** to reach the goal?
 - a) get a character, b) increment a counter
- Can use same bounds, but have different goal

Slide 3 

A Simple Example

Duration: 00:01:03

Advance mode: Auto

Notes:

Here's the example problem statement that we'll try to solve in LoopBuilding.java when you complete the charsInFirstSentence() method. We'll use this to examine the difference between the goal and the bounds

Count the characters in a string until a period is encountered. Every string will contain at least one period.

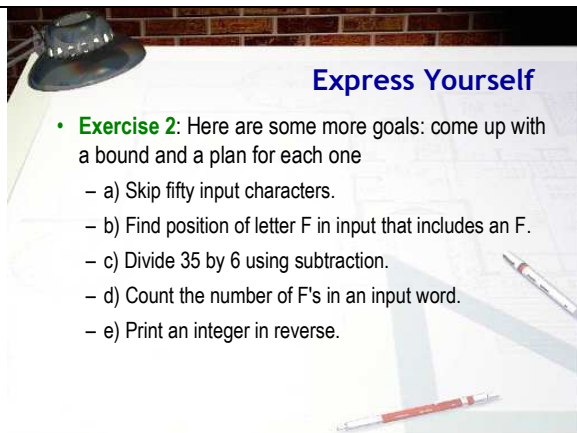
Using this problem statement, you'll find that

- the goal of the loop is to count the characters that precede a period.
- the bounds of the loop are "a period was encountered."
- the plan is to a) "read" a character, and b) increment a counter

We can use the same bounds with a different goal:

Print each character in a string until a period is encountered.

Here, the bounds is the same, but the goal is different. We're not counting, we're printing. Notice that our plan would change as well. Instead of adding one to a character, in step b), we'd simply print the character.



Express Yourself

- Exercise 2:** Here are some more goals: come up with a bound and a plan for each one
 - a) Skip fifty input characters.
 - b) Find position of letter F in input that includes an F.
 - c) Divide 35 by 6 using subtraction.
 - d) Count the number of F's in an input word.
 - e) Print an integer in reverse.

Slide 4 

Express Yourself

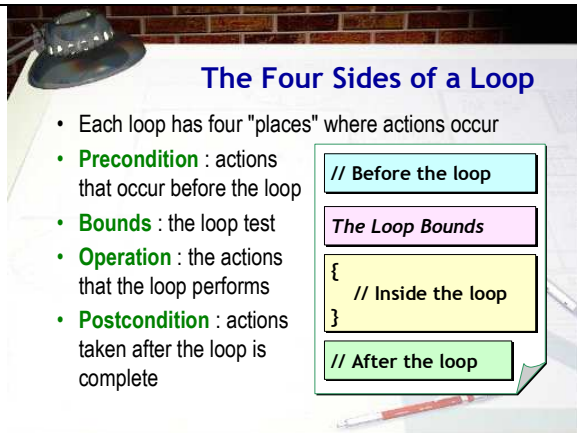
Duration: 00:01:03

Advance mode: Auto

Notes:

Here are some more examples from Cooper's book. See if you can come up with a bound and a plan for each of these goals. Just write them out, in a sentence or two after each goal. Remember the bounds is just "what happened to make the loop start and stop" and the plan is the actions you're going to carry out each time the loop repeats itself.

- Goal: Skip fifty input characters.
- Goal: Find the position of the letter F in input that includes an F.
- Goal: Divide 35 by 6 using subtraction.
- Goal: Count the number of F's in an input word.
- Goal: Print an integer in reverse.



The Four Sides of a Loop

- Each loop has four "places" where actions occur
- Precondition** : actions that occur before the loop
- Bounds** : the loop test
- Operation** : the actions that the loop performs
- Postcondition** : actions taken after the loop is complete

```
// Before the loop
The Loop Bounds
{
    // Inside the loop
}
// After the loop
```

Slide 5 🎧
The Four Sides of a Loop
 Duration: 00:00:52
 Advance mode: Auto

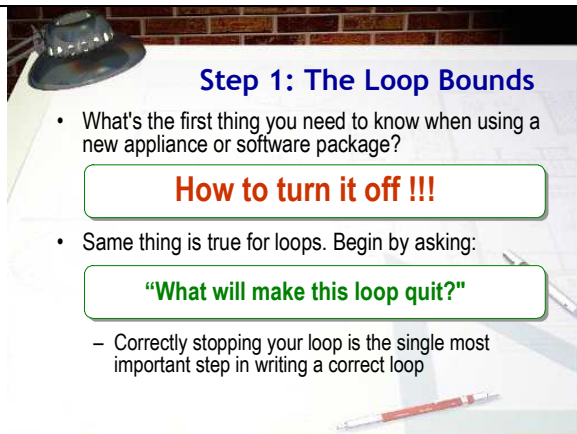
Notes:

Now that you have a "plan", you might be wondering how to put that into action; what tactics do you use to implement your overall strategy? We'll start by looking at the overall topology of a loop.

If you put a loop inside a program, as we've done with this top-tested loop, you can see that there are

- actions that occur before the loop is encountered
- a test that determines the loop's bound
- actions that occur inside the loop body
- actions that occur after the loop is complete

These four "faces" of a loop are called its precondition, its bound, its action or operation, and its postcondition. Remember these terms, because we'll use them to determine exactly where to start working on our loop.



Step 1: The Loop Bounds

- What's the first thing you need to know when using a new appliance or software package?

How to turn it off !!!

- Same thing is true for loops. Begin by asking:

"What will make this loop quit?"

- Correctly stopping your loop is the single most important step in writing a correct loop

Slide 6 🎧
Step 1: The Loop Bounds
 Duration: 00:00:41
 Advance mode: Auto

Notes:

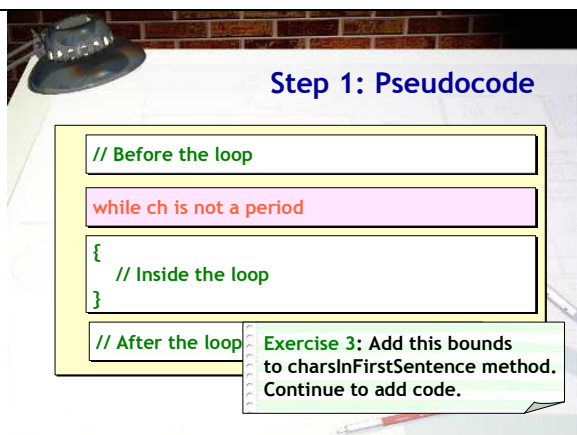
When writing a loop, step 1 is always to determine the loop bounds. If you've ever bought a new appliance, chain-saw or a new software package, what's the first thing you need to learn?

How to turn it off!!!

That's where you should always start writing your loops. Begin by asking yourself:

What will make this loop quit?

and then write the boolean expression that accomplishes that. Making sure your loops can quit is the single most important thing that you can do to make sure your loop condition is correct.



Step 1: Pseudocode

```
// Before the loop
while ch is not a period
{
    // Inside the loop
}
// After the loop
```

Exercise 3: Add this bounds to charsInFirstSentence method. Continue to add code.

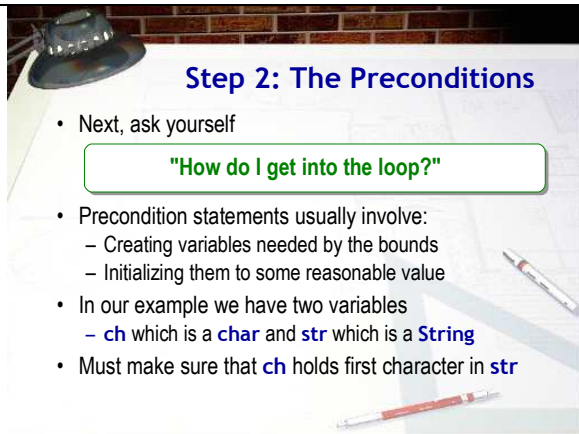
Slide 7 🎧
Step 1: Pseudocode

Notes:

Let's take our first example in the previous section, and write the loop bounds in pseudocode. (Pseudocode is simply writing a program using "English-like" phrases instead of a particular programming language.) In pseudocode, we'd say that we want to continue while ch is not a period.

For Exercise 3 let's start adding code to LoopBuilding.java. Locate the method named charsInFirstSentence() and add this bounds condition using Java code, rather than pseudocode. Don't forget to add the necessary braces around your loop body as well. Don't be alarmed if your code doesn't yet compile. We'll work on that in the next two steps. Shoot me a screen-shot of your code at this point.

Duration: 00:00:53
Advance mode: Auto



Step 2: The Preconditions

- Next, ask yourself
- "How do I get into the loop?"
- Precondition statements usually involve:
 - Creating variables needed by the bounds
 - Initializing them to some reasonable value
- In our example we have two variables
 - `ch` which is a `char` and `str` which is a `String`
- Must make sure that `ch` holds first character in `str`

Slide 8

Step 2: The Preconditions

Duration: 00:01:23
Advance mode: Auto

Notes:

Step 1, writing the bounds, makes sure that it is possible to get out of the loop. When you write this portion you'll have to make sure your boolean condition is not unavoidable or impossible, just as you did with selection statements.

Now that you've convinced yourself that it's possible to exit the loop, you have to write the statements that make it possible to enter the loop. Step 1 asks "How do I get out?", while Step 2 asks "How do I get in?"

Take a look at your bounds condition:

While `ch` is not a period

and ask yourself:

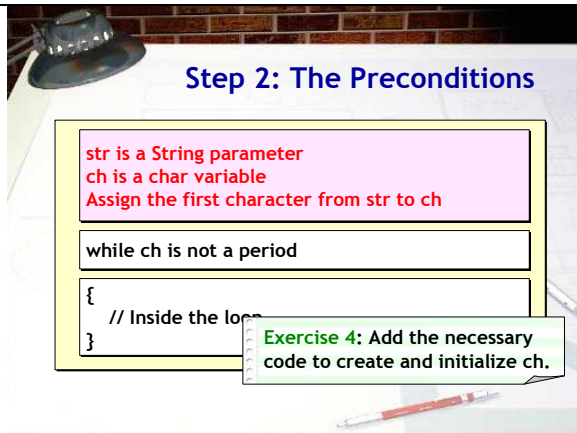
*what is `ch`?

*where did it come from?

*how did it get a value that I can check?

Of course, your compiler's already asked you essentially the same question when it refused to compile your code. Precondition statements usually involve creating variables and initializing them to some reasonable state. In our example, we have two variables implied by the problem statement, `ch`, which is a character, and `str` which is a string.

Before you encounter the loop bounds, you must make sure that `ch` exists and that it holds one of the characters in `str`; otherwise, you have no assurance that you will ever enter the loop--the value in `ch` will be unknown.



Step 2: The Preconditions

```

str is a String parameter
ch is a char variable
Assign the first character from str to ch

while ch is not a period
{
    // Inside the loop
}

```

Exercise 4: Add the necessary code to create and initialize ch.

Slide 9

Step 2: The Preconditions

Duration: 00:00:30

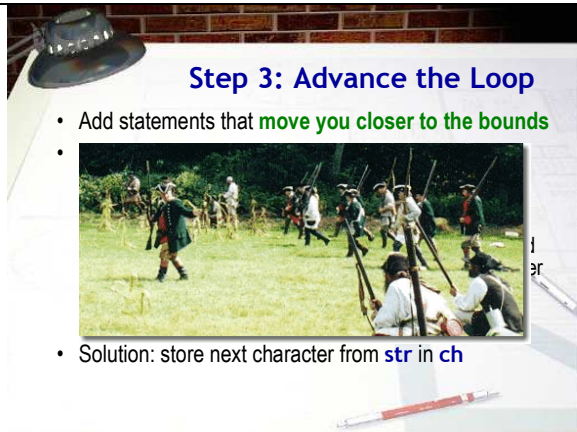
Advance mode: Auto

Notes:

In pseudocode, we could write this as:

- str is a String parameter
- ch is a char variable
- Assign the first character from str to ch

For Exercise 4, add the necessary Java code to create and initialize ch correctly. Then, save your file. All of the compiler errors should go away. Shoot me a screen-shot of your code at this point.



Step 3: Advance the Loop

- Add statements that **move you closer to the bounds**
- 
- Solution: store next character from **str** in **ch**

Slide 10

Step 3: Advance the Loop

Duration: 00:01:12

Advance mode: Auto

Notes:

Now that you've written the bounds and preconditions, it's time to advance the loop. Advancing the loop means that you add statements to the loop body that move you closer toward the loop bounds just like these Revolutionary War reenactors advanced on the redcoats.

Suppose, for instance that we didn't put any statements inside the loop body. What would happen?

If the sentence began with a period, the loop would not be entered and the program would perform correctly.

Otherwise, when the loop body was entered, nothing in the body would change the value of ch, so you would have no way out of the loop; it would simply repeat over and over, endlessly. Creating such endless or infinite loops is a very, very common error.

To avoid an endless or "dead" loop, you must make sure that the statements on the inside of the loop body change something that is tested in the loop bounds.

In this case, that's as simple as storing the next character in the sentence in the current-character.

Step 3: Advance the Loop

Exercise 5: Add the necessary code to assign the next character in str to ch.

```

str is a String parameter
ch is a char variable
Assign the first character from str to ch

while ch is not a period
{
    Assign next character in str to ch
}

```

Slide 11

Step 3: Advance the Loop

Duration: 00:01:15

Advance mode: Auto

Notes:

Here's the pseudocode for advancing the loop. Of course, this part might not be as simple when it comes to writing Java code. For Exercise 5, I want you to add the necessary code to assign the next character in str to the variable ch inside the loop body. Since the first position was at str.charAt(0), the next letter will be at str.charAt(1). The only problem is, what happens when we have to go through the loop a third time. We can't just load the second character into ch again; we want to load the third, the fourth and so on, until the loop bounds is encountered.

To solve this, that means you'll need to add another int variable, named something like pos, and initialize it to 0. Inside the loop, increment the pos variable, and then use it to access the specific character inside your string. That way, the next time you go through the loop, pos will increment to 3, then to 4, then to 5 and so on.

Once you've completed the loop, shoot me a screen-shot of your code.

Loop-Building Part I Review

- **Step 1:** Write the loop bounds
 - Check for unavoidable and impossible conditions
 - Do we have any **necessary** bounds to think about?
- **Step 2:** Write the loop preconditions
 - Add and initialize any variables used in the test
- **Step 3:** Advance the loop
 - Write statements in the loop body that move you toward the loop bounds
- Ensure the **loop works** before **putting it to work**
- **Exercise 6:** compile and run. Shoot a screen-shot.

Slide 12

Loop-Building Part I Review

Duration: 00:02:26

Advance mode: Auto

Notes:

So, let's review the first three steps in the loop-building process:

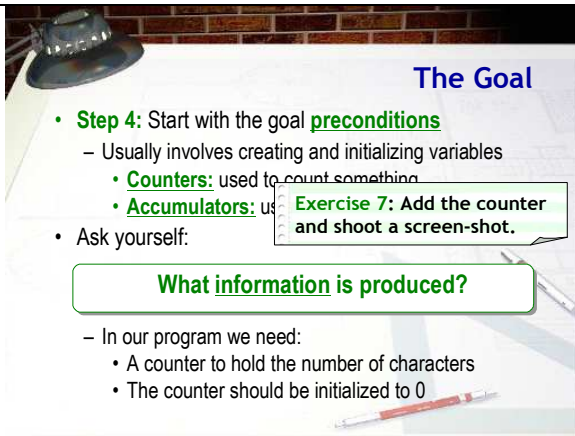
- Step 1 always starts at the loop bounds. You need to concentrate on the condition that will cause the loop to end and write the necessary code to express that. This is also the point where you need to carefully check for unavoidable and impossible conditions if your boolean condition involves AND or OR operators. Finally, you should ask yourself if you've considered all possibilities. Although I haven't asked you to do it for this exercise, your loop would be much more robust if you considered the possibility that str may not have a period, and added a necessary bounds to exit the loop once all of the letters in str had been examined.
- Step 2 always involves the bounds preconditions. Read the bounds condition and then make sure you've created and initialized any values used as part of the bounds. Initialization in this case doesn't just mean to give the variables any-old-value, but give them the correct starting value for your loop. Think Step 1: ending the loop, Step 2: starting the loop.
- Finally, Step 3 is to consider what statements are necessary inside the loop body to move the loop (potentially) closer to the bounds on the next repetition. Again, look at the bounds condition; if part of the bounds condition is a variable, (like ch), you'll have to do something to that variable in the body to move your loop forward.

At this point, the mechanical portion of your loop—the part that

makes it "work", so to speak—is finished. You should be able to compile your code, and it should run correctly. Let's see if that's right. For Exercise 6, I want you to compile and run your program and shoot me a screen-shot.

So, does it work correctly? Well, yes, if the definition of correctly is "goes around the right number of times." But really, the whole purpose of a loop is to get some real work done, to accomplish the goal. Up until now, we've ignored the goal portion entirely, because we want to make sure the loop works, before we put it to work.

Let's look at the goal now.



The Goal

- **Step 4:** Start with the goal **preconditions**
 - Usually involves creating and initializing variables
 - **Counters:** used to count something
 - **Accumulators:** used to accumulate something
- Ask yourself:

What information is produced?

 - In our program we need:
 - A counter to hold the number of characters
 - The counter should be initialized to 0

Exercise 7: Add the counter and shoot a screen-shot.

Slide 13

The Goal

Duration: 00:01:59

Advance mode: Auto

Notes:

When writing the code to carry out the goal of the loop you follow a different order than you did while working on the loop bounds. When concentrating on the loop's goal, you'll:

- start in the precondition area,
- move to the operations in the loop body,
- and then deal with the postcondition.

You didn't have to worry about the postcondition region while programming the mechanical operation of the loop, but it is often significant when it comes to the loop's goal.

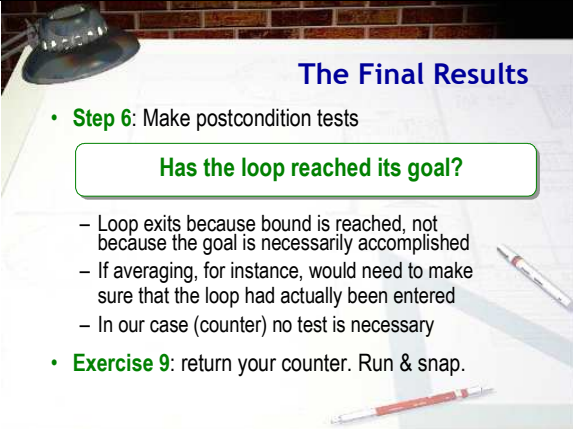
Let's look at each of these steps in a little more detail, starting with Step 4: the goal preconditions.

The goal of every loop (like the goal of every program) is to produce information of some sort or to carry out some useful modification of a piece of data, such as increasing the price of all of the products in your database. (I guess you could have a loop that was designed just to take up some processor time, but I won't consider those kinds of loops here).

Many common loops involve two kinds of calculations: counting things and adding up or otherwise processing different quantities. To do that, you'll often create special kinds of variables called counters and accumulators. An accumulator is just a name used for a variable that stores a sum of some sort.

In the precondition area of your loop, you'll create and initialize the variables necessary to carry out the goal of the loop. The secret is to ask yourself: What information is produced? Then, ask yourself what variables you'll need to store that information.

In our case, the goal is to count the characters in the first sentence, so we need a counter variable initialized to zero. For Exercise 7, go ahead and create the counter variable and shoot me a screen-shot of your code at this point.

 The Operations • Step 5: Perform necessary operations in body • Look at Step 4, and ask yourself: How is the data processed? – Usually involves updating variables or making tests – In our case, we need to update our counter each time we go through the loop • Exercise 8: update your counter. Snap a picture. Slide 14 🎧 The Operations Duration: 00:00:56 Advance mode: Auto	Notes: Step 5 is performing the necessary operations in the loop body. Of course, because a loop can have many different goals, its very hard to generalize about this step. A good way to start, though, is to look at the variables you created in Step 4, and ask: How is the data processed? Are you calculating a sum, counting different occurrences, or transforming existing values into new values? The statements inside the loop body will almost always update the variables created in the precondition in some way. For this example, we merely need to increment our counter every time through the loop, since each time we read a character (that isn't a period), we want to count it. For Exercise 8, update your counter and snap a picture of your code.
 The Final Results • Step 6: Make postcondition tests Has the loop reached its goal? – Loop exits because bound is reached, not because the goal is necessarily accomplished – If averaging, for instance, would need to make sure that the loop had actually been entered – In our case (counter) no test is necessary • Exercise 9: return your counter. Run & snap. Slide 15 🎧 The Final Results Duration: 00:01:31 Advance mode: Auto	Notes: The last step is the post-condition test. In the postcondition, you often must check one of the bounds conditions to see how to proceed. Before you leave the loop you need to ask: Have I reached my goal? The loop exists because you reached the loop bounds. Just because you reached the loop bounds doesn't mean that you've reached your goal. Let me give you a short example. Suppose you were writing a loop to calculate the average of all the positive numbers entered by the user. For your bounds condition, you had the user enter 0 or a negative number. When the loop terminates, you can't immediately calculate an average; instead, you'll have to first check to see if the user actually entered some positive numbers; if the user immediately terminated the loop by entering a negative number, attempting to calculate an average would result in a "divide by zero" error. In our case, we don't need to check the postcondition because, if we never enter the loop, our counter will be zero, which is the correct answer. We do have one final post-condition step, though, and that is to change the return statement so that it returns the value of our counter, instead of

	just 0. Once you've done that, for Exercise 9, run your program and type in a sentence. Shoot me a screen-shot of the results (which should be correct.).
Slide 16 🎧 Express Yourself Duration: 00:01:11 Advance mode: Auto	Notes: Well, that's it for the loop-building strategy. As I said, I've found this technique very useful helping me to build correct loops on the first try. After your loop is finished, though, you might want to polish it up a little. Here are some final challenges that you can complete on your own. • First, see if you can use fewer variables. In reality we have two different variables doing almost exactly the same thing. • Second, add a necessary condition so that Strings without a period don't crash your method, like they do now. Change the function so that a String without a period will exit the loop and return -1. This will probably require you to add a post-condition test, and you may need to think about it a little bit. Once you've made these two changes, test your loop with both sentences containing a period and those without, making sure you get the expected results each time. Shoot me a screen-shot of each test and past the finished code for your charsInFirstSentence() method into your exercise document.